

What is Out-of-Bag (OOB) Samples and OOB Score in Random Forest ?

1. What is Bootstrap Sampling?

Random Forest trains each Decision Tree using bootstrap sampling.

Bootstrap sampling means => Random Sampling with replacement

If the training dataset has:

N = 1000 rows and: bootstrap = True , max_samples = None

then each Decision Tree receives: 1000 random draws from the original 1000 training rows.

Because sampling is done with replacement:

- some observations may appear once,
- some observations may appear multiple times,
- some observations may never be selected.

For example, a bootstrap sample may look like: [15, 71, 230, 15, 804, 71,]

Row 15 and Row 71 have been selected more than once.

2. What is an Out-of-Bag (OOB) Sample?

For a particular Decision Tree, any training observation that was not selected at all in that tree's bootstrap sample is called an: Out of Bag (OOB) sample

For example: Original training data: Rows 1, 2, 3, ..., 1000

Tree 1 bootstrap sample: 1000 draws with replacement

Suppose Row #15 was never selected.

Then: Row #15 is OOB for Tree 1

However, Row #15 may be selected by another tree.

Therefore, **OOB status is always relative to a particular tree.**

For example, Row #15 may be:

Tree 1 → Row #15 selected → In-bag

Tree 2 → Row #15 not selected → OOB

Tree 3 → Row #15 not selected → OOB

Tree 4 → Row #15 selected → In-bag

Tree 5 → Row #15 selected → In-bag

.....

Therefore, the same observation can be:

- an **in-bag sample** for some trees, and
- an **OOB sample** for other trees.

This is what makes OOB evaluation possible.

So, it is incorrect to simply say: "Row #15 is an OOB observation of the entire forest."

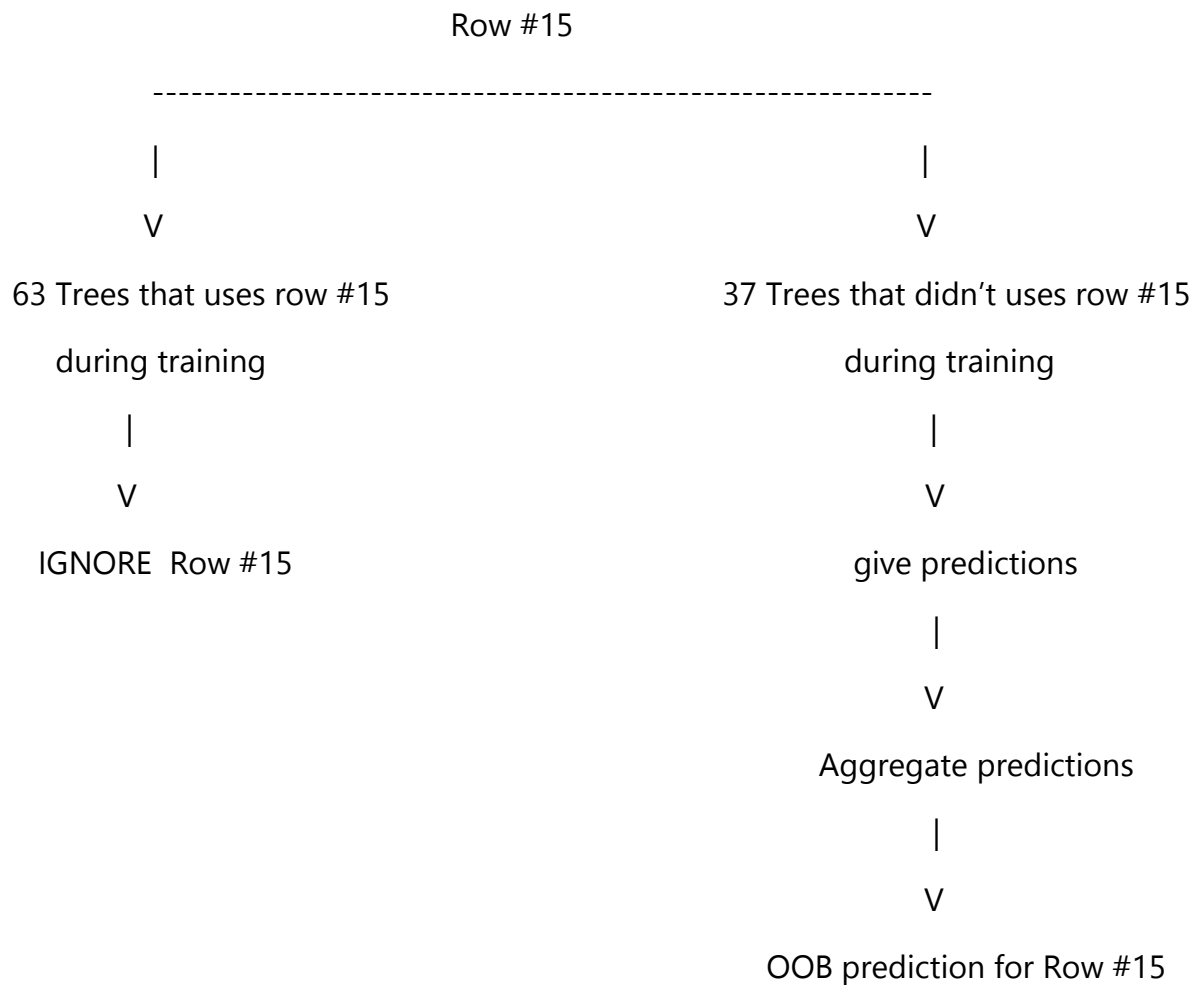
The correct statement is: "Row #15 is OOB for certain trees."

3. How OOB Prediction Is Made for One Observation

Suppose Row #15 is:

- In -bag for 63 trees
- OOB for 37 trees

Conceptually:



The important rule is

Trees that trained on Row #15 are excluded from its OOB prediction.

Is Row #15 unseen by the whole Random Forest? No

4. OOB prediction is computed separately for every training observation

For: N, training samples it produced :

$(x_1, y_1) \Rightarrow y_{1_hat_OOB}$

$(x_2, y_2) \Rightarrow y_{2_hat_OOB}$

$(x_3, y_3) \Rightarrow y_{3_hat_OOB},$

$(X_n, y_n) \Rightarrow y_{n_hat_OOB},$

Each observation has its own set of eligible OOB trees.

For example:

Row 1 $\rightarrow$ Trees {2, 5, 8, 10, ...}

Row 2 $\rightarrow$ Trees {1, 4, 6, 11, ...}

Row 3 $\rightarrow$ Trees {3, 5, 7, 9, ...}

Row N $\rightarrow$ its own collection of OOB trees

Therefore, the OOB prediction mechanism is performed **observation by observation.**

5. Does another tree see Row #15 cause leakage into its OOB prediction?

No, not merely because another tree saw the observation.

Each tree is fitted separately on its own bootstrap sample.

If: Tree 1 trained on Row #15, but Tree 2 did not. Tree 1 does not somehow transfer Row #15's training information into Tree 2.

And when Row #15's OOB prediction is calculated: Tree 1 is excluded.

Therefore, ordinary overlap among bootstrap samples should **not** be described as indirect data leakage into the observation's OOB prediction.

A statement such as: "The whole forest saw the row somewhere, therefore information leaks indirectly into its OOB prediction." is **misleading**.

6. Why OOB Evaluation Is Considered Out-of-Sample for a Particular Observation

Suppose Row #15 was OOB for Tree 7.

Tree 7 never used Row #15 while fitting its splits and leaves. Therefore, Tree 7's prediction for Row #15 is an out-of-training-sample prediction **with respect to Tree 7**

For Row #15's OOB prediction, all trees that trained on Row #15 are excluded.

7. Classical & Sklearn OOB prediction in regression

For regression, each eligible OOB tree produces a numerical prediction.

Suppose Row #15 is OOB for K trees.

If their predictions are:

$$\hat{y}_{15}^{(1)}, \hat{y}_{15}^{(2)}, \dots, \hat{y}_{15}^{(K)}$$

then the OOB regression prediction is obtained by averaging them:

$$\hat{y}_{15}^{OOB} = \frac{1}{K} \sum_{k=1}^K \hat{y}_{15}^{(k)}$$

8. OOB prediction in Classical Random Forest Classification

In Breiman's original Random Forest formulation, for classification, only the trees for which observation is OOB are allowed to vote.

Suppose Row #15 is OOB for 37 trees

Those trees might predict:

Class 1: 25 trees

Class 0: 12 trees

Then the classical OOB Classifier predicts: $\hat{y}_{15_OOB} = 1$

because Class 1 receives more OOB votes.

9. OOB predictions in scikit-learn Random Forest Classification

For a training observation x_i , suppose:

$$\mathcal{T}_i^{OOB}$$

denotes the set of trees for which x_i was OOB.

Then conceptually:

$$P^{OOB}(c \mid x_i) = \frac{1}{|\mathcal{T}_i^{OOB}|} \sum_{t \in \mathcal{T}_i^{OOB}} P_t(c \mid x_i)$$

where:

- c = a class,
- $P_t(c \mid x_i)$ = probability assigned to class c by tree t ,
- $|\mathcal{T}_i^{OOB}|$ = number of trees for which observation i is OOB.

The predicted OOB class is then:

$$\hat{y}_i^{OOB} = \arg \max_c P^{OOB}(c \mid x_i)$$

10. OOB Prediction vs OOB Score

OOB Prediction

An OOB prediction is the prediction for **one training observation**, using only trees that did not train on that observation.

For Row i :

$$\hat{y}_i^{OOB}$$

OOB Score

After obtaining OOB predictions for the training observations, compare those predictions with their true target values.

Conceptually:

$$y_i \quad \text{vs} \quad \hat{y}_i^{OOB}$$

for all eligible observations.

A performance metric computed from these OOB predictions is the:

$$\text{OOB score}$$

11. OOB Score & Metric in scikit-learn

`RandomForestRegressor(bootstrap = True, oob_score = True)`

Classification => Default OOB metrics => Accuracy

`RandomForestClassifier(bootstrap=True, oob_score = True)`

Regression => Default OOB metrics => R2 Score

Metrics such as F1, MAE or RMSE are possible evaluation metrics, but they are not automatically the default value of `oob_score`.

12. `bootstrap=False` and OOB

If: `bootstrap=False`

then every tree uses the whole training dataset in scikit-learn.

There are therefore no observations naturally left out through bootstrap sampling.

So, the standard OOB mechanism requires `bootstrap = True`

This is why OOB scoring and bootstrap sampling are fundamentally connected in Random Forest.

13. But OOB does not protect against every kind of data leakage

OOB protects against the direct situation where an observation is used to train the same trees that evaluate it.

However, leakage can still be introduced elsewhere in the ML pipeline.

For example, suppose some preprocessing is fitted on the entire dataset before the Random Forest is trained in a way that improperly uses information that should have remained unavailable during evaluation.

14. OOB evaluation is an internal estimate

OOB evaluation uses the training dataset itself but cleverly obtains predictions for each observation only from models that did not train on that observation.

Therefore, it is often described as an:

Internal estimate of generalization performance

Scikit-learn explicitly describes OOB error as being computed from trees whose bootstrap samples do not contain the particular training observation.

Breiman's Random Forest material similarly describes OOB as an internal estimate of classification/generalization error.

15. OOB Score vs Training Score

These must not be confused.

Training score

If the forest predicts observations using trees that may have trained on those same observations:

This can be overly optimistic because the model is being evaluated on data it trained on.

OOB score

For every training observation, only trees that did not train on that observation contribute to its OOB prediction.

Provides a much more meaningful estimate of generalization performance than ordinary training accuracy/error.

16. OOB Score vs Test Score

OOB evaluation and a separate test set are not the same thing.

OOB evaluation

- originates from the training dataset,
- uses bootstrap exclusion,
- gives each training observation predictions from trees that did not train on it,

Independent test set

The test observations are completely held out from training and model fitting, assuming the workflow is correctly designed.

Therefore: **OOB score != Test Score**

Their numerical values might occasionally be equal, but they represent different evaluation mechanisms.

17. Is OOB always optimistic or always pessimistic?

Neither is universally true.

Breiman's original analysis discusses conditions under which OOB estimates provide useful and effectively unbiased error estimation after enough trees are grown and notes that when too few trees contribute to each OOB prediction, the OOB error may be higher because each OOB prediction uses only a subset of the forest.

18. Why OOB can be computationally useful

Without OOB, one common validation approach would be:

Training data => training subset + validation subset

OOB allows Random Forest to obtain an internal validation-style estimate while still using bootstrap samples from the available training data.

Therefore, one advantage is: No separate validation split is required just to obtain OOB estimate

Breiman specifically emphasized OOB estimation as a way to obtain an internal estimate of generalization error.

19. Does OOB eliminate the need for a final test set?

There are two contexts to distinguish.

Breiman's original Random Forest paper argued, based on his methodology and empirical results, that OOB error could provide a highly useful internal estimate and could remove the need to set aside data solely for estimating the Random Forest's error.

However, in a modern ML workflow—especially when you are comparing many models, repeatedly tuning hyperparameters, selecting features, or making methodological choices based on OOB performance—keeping a genuinely untouched final test set remains good practice for an independent final evaluation.

For practical ML work:

OOB is excellent for internal evaluation, but an untouched test is valuable for final model assessment

20. What if there are too few trees?

With a very small number of trees, it is possible that a particular training observation is never OOB for any tree.

For such an observation, there are no eligible trees from which to form an OOB prediction.

Scikit-learn warns that with too few estimators, some observations may never be left out during bootstrap sampling, and the resulting OOB decision function can contain missing (NaN) values.

Therefore: Use enough trees if you want stable OOB estimates

Q2. Why do we say that approximately 36.8% of Observations are considered as OOB samples and approximately 63.2% of Original Observations appear in a Bootstrap Sample ?

Assume: N training observations in original data D

Set: standard bootstrap sampling

|
V

max_samples = None , bootstrap = True

Therefore, each tree performs: N bootstrap draws from the N original observations of original data D

Random Experiment : N bootstrap draws from the N original observations of original data D

Step 1: Start with ONE draw

Consider one observation such as Row #15 & single decision tree

We define:

Event A : Row # 15 appears 1 time in one draw

Event B : Row # 15 appears 0 time in one draw

They are mutually exclusive event , meaning they cannot both occur at the same time

Therefore: $P(A) + P(B) = 1$

P (Row # 15 appears 1 time in one draw) = $1/N$

P (Row # 15 appears 0 time in one draw) = $1 - 1/N$

Step 2: Move from ONE draw to ONE tree

Since there are N independent bootstrap draws with replacement for one tree

What is the probability that Row #15 is never selected in any of those 1000 draws ?

For Row #15 to be OOB, it must be missed on:

Draw1

Draw2

Draw3

.....

Draw 1000

Which are total N draws

For Row #15 to be OOB, it must be missed on every one of the N draws

Consider:

$L = P(\text{Row \#15 is never selected in complete bootstrapped data of N draws})$

$M = P(\text{Row \# 15 is OOB for the tree})$

Since $L = M$

Therefore :

$P(\text{Row \# 15 is OOB for the one tree}) = (1 - 1/N) * (1 - 1/N) * \dots N \text{ times}$

$P(\text{Row \# 15 is OOB for the one tree}) = (1 - 1/N) ^ N$

$P(\text{Row \# 15 is OOB for the one tree}) = (1 - 1/1000) ^ 1000$

$P(\text{Row \# 15 is OOB for the one tree}) \sim 0.368$

This means: Row #15 has about a 36.8% chance of appearing 0 times in that tree's bootstrap sample.

Therefore: $P(\text{a particular row is OOB for a one tree}) \sim 0.368$

Step 3: Calculate probability of being In-Bag

There are only two possibilities for Row #15: either OOB or In-Bag

So: $P(\text{In-Bag}) = 1 - P(\text{OOB})$

$P(\text{Row \# 15 is In-Bag for the one tree}) = 1 - 0.368$

Therefore:

$P(\text{a particular row is In-Bag for a one tree}) \sim 0.632$

$P(\text{a particular row is OOB for a one tree}) \sim 0.368$

Step 4: Move from 1 ROW to ALL 1000 ROWS for single Tree

Now stop focusing only on Row #15.

For one tree, out of all $N = 1000$ original rows, on average what fraction do we expect to be OOB sample ?

Here comes Expected Value :

Expected value represents the average outcome of a random experiment over many trials

$E(K) = \text{Summation } [x * P(x)]$

x = each possible outcome

$P(x)$ = probability of that outcome

The expected value ($E(K)$) is the average outcome you expect to see if you repeat a random experiment many times.

Suppose:

$X_i = 1$, if a row i is an OOB observation

$X_i = 0$, if a row i is an In-Bag observation

K = From what fraction do we expect to be OOB sample ?

Therefore, the expected number of rows that are OOB sample for single tree is:

$$E[K] = X_1 * P(X_1) + X_2 * P(X_2) + X_3 * P(X_3) + \dots + X_{1000} * P(X_{1000})$$

$$E[K] = 1 * 0.368 + 1 * 0.368 + \dots + 1 * 0.368$$

$$E[K] = 1000 * 0.368 = 368$$

$$E[\text{number of OOB row for one tree}] = 368$$

$$\text{And as percentage} = (368/1000) * 100 = 36.8\%$$

One important detail: this is an **expected percentage**, not an exact guarantee.

Tree 1 might have 355 OOB rows,

Tree 2 might have 371 OOB rows,

Tree 3 might have 362 OOB rows and so on.

Finally:

Expected number of rows that are OOB for that tree ~ 36.8%

Expected number of rows that are In bag for that tree ~ 63.2%

P (a particular row is In-Bag for a one tree) ~ 0.632

This does not mean that the tree receives only 63.2% of N rows.

For: N = 1000 & max_samples = None,

the tree still receives: N = 1000 bootstrap draws

but because some observations repeat, only approximately only around 632 of the original observations are expected to be represented

The remaining approximately: 368 original observations are expected to be OOB for that tree.

Therefore: **1000 bootstrap draws != 1000 unique observations**

STEP 5: Bernoulli Interpretation

For individual tree :

P (a particular row is OOB for a one tree) $\sim 0.368 \Rightarrow$ Success

P (a particular row is In-Bag for a one tree) $\sim 0.368 \Rightarrow$ Failure

For one single tree: It can be treated as it belongs to Bernoulli distribution

Bernoulli distribution: Is a probability distribution that models binary outcomes, where the outcome can be either Success or Failure

STEP 6: Binomial Interpretation

Suppose the Random Forest contain $T = 100$ trees

This can be treated as it belongs to Binomial Distribution

Binomial Distribution: Is a probability distribution that describes number of Success in a fixed number of independent Bernoulli trials with two possible outcomes either Success or Failure, where the probability of success is constant for each trial

Therefore:

The number of trees for which Row #15 is OOB follows a Binomial distribution $\sim \text{Binomial}(n, p) \sim \text{Binomial}(100, 0.368)$

The expected value of a Binomial random variable is:

$$E[K] = np$$

$$E[K] = 100 * 0.368 \sim 36.8$$

Thus Row #15 is expected to be OOB for approximately 37 trees out of 100 trees

STEP 7: Mathematical Meaning of Expected Value

For each tree:

P (a particular row is OOB for a one tree) ~ 0.368

Suppose the Random Forest contains $t = 100$ trees.

For each tree, define:

$X_t = 1$, if Row #15 is OOB for Tree t

$X_t = 0$, otherwise

K = the expected number of trees for which Row #15 is OOB

$$E[K] = X_1 * P(X_1) + X_2 * P(X_2) + X_3 * P(X_3) + + X_{100} * P(X_{100})$$

$$E[K] = 1 * 0.368 + 1 * 0.368 + + 1 * 0.368$$

$$E[K] = 36.8$$

Hence, we usually say:

Row # 15 is expected to be OOB for approximately 37 trees on average

The statement: $E[K] = 36.8 \Rightarrow$ does not mean that every 100-tree Random Forest will have Row #15 OOB for exactly 37 trees.

In an actual Random Forest,

Row #15 might be OOB for: 32, 35, 37, 36, 41 trees.

Expected value means:

If we repeatedly build many Random Forests with 100 trees, the average number of trees for which Row #15 is OOB will approach approximately 36.8

It does not mean that forest must contain exactly 36.8 or 37 OOB trees.

Expected value = long run average, not a guaranteed outcome

Why the Number Is Not Exactly 37 ?

Probability does not guarantee an exact outcome.

For example, a fair coin has:

$P(\text{Head}) = 0.5 \Rightarrow \text{Success}$

$P(\text{Tail}) = 0.5 \Rightarrow \text{Failure}$

If we toss it $n = 100$ times, $X = \{H, T, H, H, T, H, T, T, H, \dots, 100\}$

then this X also follows Binomial Distribution

$X \sim \text{Binomial}(n, p) \sim \text{Binomial}(100, 0.5)$

Expected number of heads is: $100 * 0.5 = 50$

But the actual result might be: 47, 52, 55, 49

Likewise,

for OOB sampling: $P(\text{OOB}) = 0.368$

so, the expected number among 100 trees is: 36.8.

but random variation causes the actual number to fluctuate around 36.8.

But one actual Random Forest might have Row #15 OOB for: 32 trees, 35 trees, 37 trees, 40 trees, 41 trees and so on

Expected value = long run average, not a guaranteed value

STEP 8: Standard Deviation — Why values like 32 or 41 are normal

The expected value tells us the **centre** of the distribution:

$$\mu = 36.8$$

But it doesn't tell us how much the actual number tends to fluctuate.

For a Binomial random variable: $K \sim \text{Binomial}(n, p)$

the variance is:

$$\text{Var}(K) = np(1 - p) = 100 * 0.368 * (0.632) \sim 23.24$$

$$\text{Standard deviation} = (23.24)^{1/2} \sim 4.82$$

So, the distribution is centred around 36.8 with a typical fluctuation of roughly 4.82 trees

For example:

$$36.8 - 4.82 \sim 32 \text{ trees}$$

$$36.8 + 4.82 \sim 41 \text{ trees}$$

Therefore, values such as 32, 41 are quite reasonable outcomes.